Parte II · Coordinación (capítulos 6 a 13)

Notas de lectura — Vitillo, R. *Understanding Distributed Systems* (2.ª ed.) Sistemas Distribuidos · Escuela de Ingeniería de Sistemas e Informática · UIS

Propósito de la parte

La Parte I explicó cómo **comunicar** dos procesos. La Parte II aborda cómo lograr que **varios procesos cooperen** para alcanzar un objetivo común a pesar de fallas, retrasos y ausencia de un reloj global. El hilo conductor es la **replicación de datos**: para tener **alta disponibilidad** hay que mantener copias en varios nodos, y mantenerlas coherentes exige **coordinación**. La parte avanza desde los modelos teóricos (cómo razonar sobre fallas y tiempo) hasta mecanismos concretos (elección de líder, Raft, quórums, CRDT, transacciones distribuidas y sagas).

Cap.	Tema	Pregunta central
6	Modelos de sistema	¿Qué suposiciones hacemos sobre enlaces, procesos y tiempo?
7	Detección de fallas	¿Cómo saber si un proceso cayó?
8	Tiempo	¿Cómo ordenar eventos sin un reloj compartido?
9	Elección de líder	¿Cómo garantizar que un solo proceso actúe como coordinador?
10	Replicación	¿Cómo mantener réplicas consistentes? ¿Qué garantías dar al cliente?
11	Evitar la coordinación	¿Se puede replicar sin un líder y sin consenso?
12	Transacciones	¿Cómo lograr atomicidad y aislamiento entre varios nodos?
13	Transacciones asíncronas	¿Cómo coordinar servicios que no comparten base de datos?

Capítulo 6 · Modelos de sistema

Para razonar sobre algoritmos distribuidos se necesitan **modelos**: abstracciones que fijan qué puede y qué no puede pasar.

Modelos de enlace:

- **Enlace con pérdida justa** (*fair-loss*): los mensajes pueden perderse o duplicarse, pero si se reenvían suficientes veces, alguno llega.
- **Enlace confiable**: entrega cada mensaje exactamente una vez, sin pérdidas ni duplicados. Se construye sobre uno de pérdida justa con **retransmisiones y deduplicación** (TCP).
- **Enlace confiable autenticado**: además autentica al emisor (TLS).

Modelos de proceso:

- **Falla arbitraria (bizantina)**: el proceso puede desviarse del algoritmo de cualquier forma, incluso maliciosamente. Típico de blockchains o sistemas críticos (aviones).

- **Caída con recuperación** (*crash-recovery*): el proceso puede detenerse y luego reiniciarse; conserva el estado persistente, pierde el que está en memoria.
- **Caída con detención** (*crash-stop*): una vez que cae, no vuelve.

Modelos de temporización:

- **Síncrono**: hay un límite conocido para el tiempo de envío de mensajes y de ejecución de operaciones.
- **Asíncrono**: no hay límites; un mensaje o una operación pueden tardar arbitrariamente.
- **Parcialmente síncrono**: el sistema se comporta de forma síncrona la mayor parte del tiempo, pero ocasionalmente no. Es el modelo que mejor describe los sistemas reales.

> Salvo que se diga lo contrario, el libro asume **enlaces con pérdida justa, procesos con caída y recuperación y sincronía parcial**. Los modelos son simplificaciones: lo importante es conocer sus supuestos para saber cuándo se rompen.

Capítulo 7 · Detección de fallas

- Cuando un cliente envía una petición y no recibe respuesta, **no puede distinguir** entre: el servidor cayó, el servidor es lento, la petición se perdió o la respuesta se perdió.
- La única herramienta práctica es el **tiempo de espera** (*timeout*): si no llega respuesta en cierto plazo, se **sospecha** que el proceso falló.
- **Compromiso del *timeout***: si es corto, se declaran caídos procesos que solo estaban lentos (falsos positivos) y se reintenta de más; si es largo, se tarda en reaccionar ante caídas reales.
- Un **detector de fallas** es el componente que informa qué procesos parecen caídos. Dos técnicas:
  - ***Ping***: un proceso envía periódicamente una petición a otro y espera respuesta en un plazo.
  - ***Heartbeat*** (latido): el proceso vigilado envía periódicamente un mensaje "estoy vivo"; si deja de llegar, se lo declara caído.
- Los detectores de fallas son **imperfectos** por naturaleza en un sistema asíncrono: solo pueden **sospechar**.
- Se usan sobre todo cuando los procesos interactúan con frecuencia; en otros casos basta con tratar el error cuando la petición falla.

Capítulo 8 · Tiempo

Relojes físicos:

- Cada máquina tiene un reloj de cuarzo que **deriva** (se adelanta o atrasa). Se sincroniza con **NTP**, que corrige estimando la latencia de red.
- La sincronización tiene un **error** y puede hacer que el reloj **salte hacia atrás o adelante**. Por eso no se debe usar el reloj de pared (*wall clock*) para medir intervalos ni para ordenar eventos entre máquinas.
- Para medir tiempo transcurrido se usa un **reloj monótono**, que nunca retrocede (pero no sirve para comparar entre máquinas).

Relojes lógicos de Lamport:

- Capturan la relación **sucedió-antes** (*happened-before*, $\rightarrow$): $a \rightarrow b$ si están en el mismo proceso y a ocurre primero, si a es el envío y b la recepción del mismo mensaje, o por transitividad.

- Reglas: cada proceso tiene un contador; lo **incrementa** antes de cada evento local; al **enviar**, adjunta su valor; al **recibir**, toma `max(local, recibido) + 1`.
- Garantía: si a → b, entonces L(a) < L(b). **El recíproco no se cumple**: L(a) < L(b) no implica a → b (pueden ser concurrentes).
- Desempatando por identificador de proceso se obtiene un **orden total** consistente con la causalidad.

Relojes vectoriales:

- Cada proceso mantiene un **vector** con un contador por proceso. Incrementa su propia entrada en cada evento; al recibir, toma el **máximo componente a componente** y luego incrementa la suya.
- Permiten decidir si dos eventos están **causalmente relacionados** o son **concurrentes**: V(a) < V(b) si y solo si a → b. Si ninguno es menor, son concurrentes.
- Costo: el tamaño crece con el número de procesos.

Capítulo 9 · Elección de líder

Muchas tareas requieren que **un solo proceso** actúe como coordinador (por ejemplo, el único que acepta escrituras). La elección de líder debe garantizar:

- **Seguridad** (*safety*): como máximo **un líder** a la vez.
- **Vivacidad** (*liveness*): eventualmente se elige un líder, incluso con fallas.

Elección en Raft:

- Cada proceso está en uno de tres estados: **seguidor**, **candidato** o **líder**.
- El tiempo se divide en **mandatos** (*terms*), numerados de forma creciente; cada mandato tiene a lo sumo un líder.
- El líder envía *heartbeats* periódicos. Si un seguidor no recibe nada durante un **tiempo de elección**, se convierte en candidato, **incrementa el mandato**, vota por sí mismo y pide votos.
- Cada proceso vota **una sola vez por mandato**. Quien obtiene la **mayoría** es líder. Si nadie la obtiene (votos divididos), se repite con un nuevo mandato; los tiempos de elección **aleatorios** hacen poco probable que se repita el empate.
- Si un candidato o líder descubre un mandato mayor, vuelve a seguidor.

Consideraciones prácticas:

- En la práctica rara vez se implementa el algoritmo desde cero: se usa un almacén **linealizable** externo (etcd, ZooKeeper, DynamoDB) con un **arrendamiento** (*lease*): una clave con **TTL** que se adquiere con una operación atómica (*compare-and-swap*) y se renueva periódicamente.
- Peligro: un líder puede creer que aún tiene el arrendamiento cuando ya expiró (por ejemplo, por una **pausa** de recolección de basura). Solución: **fichas de cercado** (*fencing tokens*): cada adquisición devuelve un número de versión creciente y el recurso rechaza operaciones con una ficha antigua.
- Un líder es un **punto único de falla** y un **cuello de botella**: conviene que haga el menor trabajo posible y, si se puede, evitarlo (cap. 11).

Capítulo 10 · Replicación

Replicación de máquinas de estado con Raft:

- Si cada réplica parte del mismo estado y aplica **las mismas operaciones en el mismo orden**, todas llegan al mismo estado. El problema se reduce a mantener un **registro** (*log*) idéntico en todas: es el problema del **consenso**.
- El líder recibe las operaciones, las agrega a su registro y las envía a los seguidores (`AppendEntries`). Una entrada se **confirma** (*commit*) cuando está en la **mayoría** de las réplicas; entonces se aplica a la máquina de estados y se responde al cliente.
- Con 2f + 1 réplicas se toleran f fallas. Solo puede ser elegido líder un candidato con registro al menos tan actualizado como el de la mayoría.

Modelos de consistencia (garantías que ve el cliente):

- **Fuerte (linealizabilidad)**: cada operación parece ejecutarse atómicamente en un instante entre su invocación y su respuesta; el sistema se comporta como si hubiera una sola copia. En Raft, leer del líder exige confirmar que sigue siendo líder (o usar un arrendamiento).
- **Secuencial**: todos ven las operaciones en el mismo orden, consistente con el orden de cada cliente, pero no necesariamente en tiempo real (p. ej., leer de un seguidor que va atrasado pero en orden).
- **Eventual**: las réplicas convergen si dejan de llegar escrituras; un cliente puede leer valores viejos.
- **Teorema CAP**: ante una **partición de red**, un sistema debe elegir entre **consistencia** (fuerte) y **disponibilidad**. **PACELC** lo extiende: si hay **P**artición, elegir entre **A** y **C**; **E**n caso contrario, entre **L**atencia y **C**onsistencia.

Replicación en cadena:

- Las réplicas forman una cadena: las escrituras entran por la **cabeza** y se propagan hasta la **cola**, que confirma; las lecturas se atienden en la **cola** (consistencia fuerte).
- La configuración de la cadena la administra un **plano de control** tolerante a fallas (que sí usa consenso); el plano de datos no.
- Variante **CRAQ**: permite leer de cualquier nodo y consultar a la cola solo si el objeto tiene escrituras pendientes.
- Ventaja: separa la gestión de configuración (consenso) del camino de datos; desventaja: la latencia de escritura crece con la longitud de la cadena y un nodo lento la frena entera.

Capítulo 11 · Evitar la coordinación

El consenso es costoso y el líder limita la escalabilidad. Este capítulo busca **replicar sin coordinación**, aceptando consistencia más débil.

- **Protocolos de difusión** (*broadcast*): *best-effort* (sin garantías si el emisor cae), **confiable** (si un proceso correcto entrega un mensaje, todos lo hacen; versión "ansiosa" con reenvío de todos a todos, o **chismes** — *gossip* — que reenvía a un subconjunto aleatorio), y de **orden total** (todos entregan en el mismo orden; requiere consenso).
- **CRDT** (tipos de datos replicados libres de conflictos): si las réplicas solo se comunican mediante una operación de **fusión** (*merge*) que es **conmutativa, asociativa e idempotente** (los estados forman un *semirretículo*), convergen sin coordinación: **consistencia eventual fuerte**. Ejemplos: registro **último escritor gana** (LWW), registro multivalor (MV), **contador creciente** (G-Counter), conjuntos.
- **Almacenes estilo Dynamo** (sin líder): cualquier réplica acepta escrituras; con N réplicas, una escritura espera W confirmaciones y una lectura consulta R réplicas. Si **W + R > N**, los conjuntos se solapan y la

lectura ve al menos una copia actualizada (aunque no da linealizabilidad en todos los casos). Para converger: **reparación en lectura** (*read repair*) y **antientropía** con árboles de Merkle.
- **Teorema CALM**: un programa tiene una implementación consistente **sin coordinación** si y solo si es **monótono** (agregar información nunca invalida conclusiones previas).
- **Consistencia causal**: el modelo más fuerte que se puede lograr manteniendo disponibilidad ante particiones: las operaciones causalmente relacionadas se ven en el mismo orden en todas partes; las concurrentes pueden verse en orden distinto.

Capítulo 12 · Transacciones

Una transacción agrupa operaciones con garantías **ACID**: **atomicidad** (todo o nada), **consistencia** (invariantes de la aplicación), **aislamiento** (las transacciones concurrentes no se interfieren) y **durabilidad** (lo confirmado no se pierde).

Aislamiento:

- Anomalías: **escritura sucia**, **lectura sucia**, **lectura no repetible** (*fuzzy read*), **lectura fantasma**.
- Niveles: lectura no confirmada, lectura confirmada, lectura repetible, **serializable** (equivale a alguna ejecución en serie) y **serializabilidad estricta** (serializable + linealizable).
- Control de concurrencia:
  - **Pesimista**: **bloqueo en dos fases (2PL)**: fase de expansión (se adquieren bloqueos) y de contracción (se liberan); riesgo de **interbloqueos**.
  - **Optimista (OCC)**: se ejecuta sin bloquear y se **valida** al confirmar; si hubo conflicto, se reintenta. Conviene con pocos conflictos.
  - **MVCC**: múltiples versiones; las lecturas usan una instantánea (*snapshot isolation*) y no bloquean escrituras.

Atomicidad:

- En un nodo: **registro de escritura anticipada** (*write-ahead log*) para deshacer o rehacer tras una caída.
- Entre nodos: **confirmación en dos fases (2PC)**: un **coordinador** pide a los participantes **prepararse** (fase 1); si todos votan sí, ordena **confirmar**, si no, **abortar** (fase 2). Es **bloqueante**: si el coordinador cae tras la preparación, los participantes quedan esperando con bloqueos tomados. Es lento y frágil, pero útil.
- **NewSQL**: bases de datos distribuidas que combinan transacciones con escalabilidad. **Spanner** replica cada partición con Paxos, usa 2PC entre particiones (con coordinadores replicados, por lo que dejan de ser punto único de falla) y **TrueTime** (relojes con incertidumbre acotada) para dar serializabilidad estricta.

Capítulo 13 · Transacciones asíncronas

Cuando los datos están en servicios o almacenes distintos que **no soportan 2PC** o no deben acoplarse, se recurre a mecanismos asíncronos que sacrifican aislamiento.

- **Patrón *outbox* (bandeja de salida)**: para actualizar la base de datos **y** notificar a otro servicio de forma atómica, se escribe el cambio y el mensaje en una tabla *outbox* **en la misma transacción local**. Un

proceso de **relevo** lee la tabla y publica los mensajes en un canal (con al menos una entrega), por lo que el **consumidor debe ser idempotente**.

- **Sagas**: una transacción distribuida se divide en una secuencia de **transacciones locales** $T_1 \dots T_n$; cada una tiene una **transacción de compensación** C_i que deshace su efecto semántico. Si T_k falla, se ejecutan $C_{k-1} \dots C_1$. Ejemplo: reservar vuelo y hotel; si el hotel falla, se cancela el vuelo.
  - Un **orquestador** coordina los pasos y persiste el estado de la saga para continuar tras una caída; los pasos y compensaciones deben ser **idempotentes**.
- **Aislamiento**: las sagas **no son aisladas**: otros pueden ver estados intermedios. Mitigación: **bloqueos semánticos** (marcar un registro como "pendiente") y diseñar las operaciones para tolerarlo.

Conceptos clave (glosario)

Término	Definición breve
Sincronía parcial	El sistema suele cumplir límites de tiempo, pero no siempre
Detector de fallas	Componente que sospecha caídas mediante *timeouts* (ping, *heartbeat*)
Sucedió-antes ($\rightarrow$)	Relación de causalidad potencial entre eventos
Reloj de Lamport	Contador escalar: $a \rightarrow b \Rightarrow L(a) < L(b)$
Reloj vectorial	Vector de contadores: detecta concurrencia
Mandato (*term*)	Época lógica de Raft; a lo sumo un líder por mandato
Arrendamiento (*lease*)	Derecho con expiración para actuar como líder
Ficha de cercado	Número creciente que permite rechazar a un líder obsoleto
Linealizabilidad	Las operaciones parecen ocurrir de forma atómica en tiempo real
CAP / PACELC	Compromisos entre consistencia, disponibilidad y latencia
CRDT	Tipo de dato cuya fusión converge sin coordinación
Quórum ($W + R > N$)	Solapamiento de escrituras y lecturas en sistemas sin líder
2PC	Protocolo de confirmación atómica con coordinador; bloqueante
Saga	Secuencia de transacciones locales con compensaciones
Outbox	Tabla para publicar mensajes atómicamente con un cambio local

---

Preguntas guía para la lectura

1. ¿Qué modelo de sistema asume el libro y por qué es razonable?
2. ¿Por qué un detector de fallas solo puede *sospechar*?
3. ¿Qué captura un reloj de Lamport que no capture un reloj físico, y qué no puede capturar?
4. ¿Cómo garantiza Raft que haya como máximo un líder por mandato?
5. ¿Qué diferencia hay entre consistencia fuerte, secuencial y eventual?
6. ¿Por qué los CRDT convergen sin coordinación?
7. ¿Por qué 2PC es bloqueante y qué alternativas existen cuando no es viable?

Conexión con otras partes

- **Parte I:** los enlaces confiables y autenticados son la base de los modelos del cap. 6; los *heartbeats* se trabajaron en los talleres.
- **Parte III:** la replicación y el particionamiento son las herramientas de escalabilidad; la replicación en cadena reaparece en el almacenamiento de archivos (Azure Storage) y la separación plano de control / plano de datos tiene capítulo propio.
- **Parte IV:** la detección de fallas y los *timeouts* se retoman en los patrones de resiliencia.